DEPARTMENT OF TECHNICAL EDUCATION

GOVERNMENT OF KARNATAKA



450 - R.E.S. POLYTECHNIC, BANGALORE

DATA STRUCTURES WITH PYTHON ACTIVITIES

Course Code: 25CS33I | Semester III | Computer Science and Engineering

Submitted By:

Name: _____

Register Number: _____

RUBRICS FOR ASSESSMENT OF ACTIVITIES AND RECORD BOOK

Sl. No.	Dimension	Unsatisfactory (1-10)	Need Improvement (11-20)	Satisfactory (21-30)	Good (31-40)	Excellent (41-50)	Student Score
01	Understanding Activity Question	Do not understand at all	Poor understanding	Little understanding	Good understanding	Excellent understanding	
02	Execution of Activities	No	Major errors	Inaccuracy	Effective but minor errors	Precise and accurate	
03	Record Book Maintenance	No quality	Major flaws	Acceptable	Good quality	High quality	
04	In time submission of record	Major sections not addressed / poor	Takes time, 20% submitted in time	50% submitted in time	80% submitted in time	100% submitted in time	
Average Marks							

DATA STRUCTURES WITH PYTHON

One Activity per Week (Week 1 to Week 13) - Aim, Algorithm and Result

Name: _____ Reg. No.: _____ Section: _____

WEEK 1: Introduction to Data Structures

Activity 1: Real-time applications of built-in and user-defined data structures

Aim:

To explore and document real-time applications of built-in and user-defined data structures.

Algorithm:

1. Start.
2. List the built-in data structures (list, tuple, set, dictionary) and the user-defined ones (stack, queue, linked list, tree).
3. For each one, find a real-time application.
4. Write the application, the operation used (traversal, insertion, deletion, searching, sorting, merging) and the reason for choosing it.
5. Record the results in a table. Stop.

Result:

Applications documented. Examples: list - music playlist; dictionary - phone contacts; set - removing duplicate entries; stack - undo in an editor; queue - printer jobs; linked list - image viewer; tree - folder structure.

WEEK 2: Sets and Tuples

Activity 2: Set operations: union, intersection, difference and symmetric difference

Aim:

To write a Python program to perform set operations.

Algorithm:

1. Start.
2. Create two sets A and B.
3. Find A | B (union) and A & B (intersection).
4. Find A - B (difference) and A ^ B (symmetric difference).
5. Print the results. Stop.

Result:

Program executed. Example: A = {1, 2, 3}, B = {3, 4}: union = {1, 2, 3, 4}, intersection = {3}, difference = {1, 2}, symmetric difference = {1, 2, 4}.

WEEK 3: Lists and Dictionaries

Activity 3: Operations on lists and dictionaries

Aim:

To write a Python program to perform operations on lists and dictionaries.

Algorithm:

1. Start.
2. Create a list of marks. Use append(), remove(), sort() and a list comprehension to find the marks above 40.
3. Create a dictionary of student name and marks.
4. Add, update and delete entries. Print using keys(), values() and items().
5. Stop.

Result:

Program executed. List and dictionary operations were performed successfully.

WEEK 4: Strings and Files

Activity 4: Read a file and process its text using string methods

Aim:

To write a Python program to read a file and process the text using string methods.

Algorithm:

1. Start.
2. Open the file in read mode and read the text.
3. Use upper(), lower() and split() to change the case and separate the words.
4. Count the number of words and vowels.
5. Write the result to a new file, close both files. Stop.

Result:

Program executed. The word count and vowel count were displayed and saved in the new file.

WEEK 5: Error and Exception Handling

Activity 5: Exception handling using try, except and finally

Aim:

To write a Python program to handle exceptions.

Algorithm:

1. Start.
2. Read two numbers in a try block and calculate a / b.
3. Use except ZeroDivisionError and except ValueError to print error messages.
4. Use finally to print 'Program ended'.
5. Stop.

Result:

Program executed. Division by zero and wrong input were handled without the program crashing.

WEEK 6: Introduction to Recursion

Activity 6: Factorial of a number using recursion

Aim:

To write a recursive Python program to find the factorial of a number.

Algorithm:

1. Start.
2. Define factorial(n).
3. Base case: if n == 0 or n == 1, return 1.
4. Recursive case: return n x factorial(n - 1).
5. Read n, call the function and print the result. Stop.

Result:

Program executed. Example: factorial(5) = 120.

WEEK 7: Getting Started with ADT and UDT

Activity 7: List ADT implemented using a user-defined class

Aim:

To implement a List ADT using a user-defined data type (class).

Algorithm:

1. Start.
2. Create class MyList with an empty list in __init__.
3. Define methods insert(x), delete(x), search(x) and display().
4. Create an object and call the methods.
5. Stop.

Result:

Program executed. The List ADT worked using the class, hiding the internal list.

WEEK 8: Stack Data Structure

Activity 8: Stack operations (push, pop, peek) and reversing a string

Aim:

To implement a stack with push, pop and peek, and use it to reverse a string.

Algorithm:

1. Start.
2. Create class Stack with an empty list.
3. push(x): append x. pop(): if the stack is empty print 'Underflow', else remove and return the last item.
4. peek() returns the top item. is_empty() checks whether the stack is empty.
5. Push each character of the string and pop all of them to get the reversed string.
6. Stop.

Result:

Program executed. Last in, first out was verified. Example: 'DATA' gives 'ATAD'.

WEEK 9: Queue Data Structure

Activity 9: Queue operations (enqueue, dequeue, display)

Aim:

To implement a queue with enqueue, dequeue and display operations.

Algorithm:

1. Start.
2. Create class Queue with an empty list.
3. enqueue(x): append x at the rear.
4. dequeue(): if the queue is empty print 'Underflow', else remove the front item (index 0).
5. display() prints all items from front to rear.
6. Stop.

Result:

Program executed. First in, first out was verified. Items were removed in the order they were added.

WEEK 10: Linked Lists: Singly Linked List

Activity 10: Singly linked list operations

Aim:

To implement a singly linked list with insertion, deletion, traversal and searching.

Algorithm:

1. Start.
2. Create class Node (data, next) and class LinkedList (head = None).
3. Insert at beginning: new node next = head, then head = new node.
4. Insert at end: go to the last node and set its next to the new node.
5. Delete a node: find it and link the previous node to the next node.
6. Traverse from head till None to print; search by comparing data.
7. Stop.

Result:

Program executed. Nodes were inserted, deleted, searched and displayed.

WEEK 11: Doubly Linked List

Activity 11: Doubly linked list operations

Aim:

To implement a doubly linked list with insertion, deletion and forward and backward traversal.

Algorithm:

1. Start.
2. Create class Node (prev, data, next).
3. Insert at beginning and end by updating the prev and next links.
4. Delete a node by linking its prev node and next node to each other.
5. Forward traversal: from head using next. Backward traversal: from tail using prev.
6. Stop.

Result:

Program executed. The list was displayed in both forward and backward directions.

WEEK 12: Introduction to Trees

Activity 12: Binary search tree: insertion, searching and traversal

Aim:

To implement a binary tree with insertion, searching and traversal.

Algorithm:

1. Start.
2. Create class Node (data, left, right).
3. Insert: if value < node, go left; else go right; place at the empty position.
4. Search: compare with the node and go left or right until found or None.
5. Print the tree using in-order traversal.
6. Stop.

Result:

Program executed. In-order traversal of the binary search tree gave the values in sorted order.

WEEK 13: Tree Traversal (Recursive and Iterative)

Activity 13: In-order, pre-order and post-order traversal

Aim:

To implement recursive and iterative tree traversals.

Algorithm:

1. Start. Create a binary tree.
2. Recursive in-order: left, root, right. Pre-order: root, left, right. Post-order: left, right, root.
3. Iterative in-order: use a stack; push the left nodes, pop, visit, then move to the right child.
4. Print the three traversals.
5. Stop.

Result:

Program executed. Example: tree 2 with children 1 and 3 gives in-order 1 2 3, pre-order 2 1 3, post-order 1 3 2.

Student Signature	Staff Signature	HOD Signature